



Ops-Management-Dashboard — The Lesson

Chain C · Project C-3 · the full written lesson, start to finish

1 — HTTP vs WebSockets

Every project so far has been request/response: the client asks, the server answers, the connection closes. A dashboard wanting "live" data from that model has exactly one option — poll, over and over, mostly asking "anything new?" and getting "no." A WebSocket is a single long-lived connection where either side can send a message at any time, with no request to trigger it. `POST /tickets` still works like C-1's API, but now it also pushes a message to whichever connected dashboards should know — the instant it happens.

2 — The ConnectionManager Pattern

`active_connections: dict[int, tuple[WebSocket, str]]` — keyed by `user_id`, storing each connection alongside the role it connected with. `broadcast(event)` walks every connection and sends only to the ones whose role matches `event.target_role`.

A real trap this project's own test fell into: the first version of "staff should NOT receive a MANAGER-targeted event" tried to prove absence by calling `receive_json(timeout=0.2)` inside a try/except. Starlette's test session doesn't accept a `timeout` argument — it raised `TypeError` immediately, every time, and the broad except swallowed it as "nothing arrived." The test passed even after deliberately breaking `broadcast()` to ignore `target_role` completely. The fix: prove ORDERING instead — trigger a second, STAFF-targeted event right after, and assert staff's first received message is that one, not the manager-targeted one.

3 — Role-Based Access Control

`require_manager` is a FastAPI dependency, not an `if` inside the route. FastAPI resolves dependencies before the route body runs, so a STAFF caller to `POST /tickets/{id}/assign` gets 403 before `TICKETS.get(ticket_id)` — or anything else in that function — ever executes. Authentication (who are you) and authorization (what are you allowed to do) stay separate, composable pieces.

Why this placement matters: an inline `if role != "MANAGER"` check, written after other logic, is one refactor away from being accidentally skipped. A dependency is structurally guaranteed to run first — there's no code path around it.

4 — Real-Time UI Updates

`useWebSocket.ts` owns the connection and reconnect logic; `WebSocketProvider.tsx` decides what an incoming event means. Its rule is deliberately broad: any `event_type` starting with

"ticket_" invalidates the ["tickets"] query — not a switch matching each exact type to a specific state patch, which would need updating every time the backend adds an event type.

The trap useWebSocket.ts guards against: `ws.onclose` fires both when the connection actually drops AND when you call `socket.close()` yourself (component unmounting). Without a `closedByEffectCleanup` flag, unmounting triggers the reconnect handler, opening a new socket for a component that no longer exists — whose eventual close schedules another reconnect, forever.